

Optimisation par Colonie de Fourmis d'un Chemin sur un Paysage Fractal

Rapport de Parallélisation (OpenMP & MPI)

Naomy Isabela GOMES BIAGGI

13 mars 2026

1 Introduction

Ce rapport présente les résultats de l'optimisation et de la parallélisation d'un algorithme de colonie de fourmis (ACO) conçu pour trouver le chemin le plus court sur un terrain fractal. L'objectif principal est de réduire le temps de calcul du déplacement des fourmis et de l'évaporation des phéromones en utilisant des techniques de parallélisation en mémoire partagée (OpenMP) et en mémoire distribuée (MPI).

2 Profilage de la version séquentielle

Avant toute parallélisation, une analyse de performance a été réalisée à l'aide de l'outil `gprof` sur la version vectorisée du code.

Les résultats du *flat profile* montrent que la majorité du temps d'exécution est concentrée sur une seule fonction :

- `ant::advance` : **61.48 %** du temps total.
- `Renderer::display` : 26.70 % (Temps lié à l'affichage graphique).
- `advance_time` : 11.24 %.

Le temps moyen par itération pour le déplacement séquentiel était de 2.48 ms. Il est donc évident que la parallélisation doit se concentrer en priorité sur la boucle de déplacement des fourmis dans `advance_time`.

3 Parallélisation en Mémoire Partagée (OpenMP)

La boucle de déplacement des fourmis a été parallélisée en utilisant la directive `#pragma omp parallel for schedule(guided)`. L'utilisation de directives `atomic` a été nécessaire pour éviter les conditions de course (data race) lors de la mise à jour des phéromones, notamment au début de la simulation lorsque toutes les fourmis se trouvent dans le nid.

3.1 Résultats de Performance

Le tableau 1 présente le temps de déplacement total des fourmis (pour 2585 itérations) en fonction du nombre de threads. Le Speedup (S) est calculé par rapport à l'exécution avec 1 thread (5781.72 ms).

Nombre de Threads	Temps Déplacement (ms)	Speedup (S)	Temps Évaporation (ms)
1	5781.72	1.00x	1200.49
2	4027.83	1.43x	1255.97
4	2595.99	2.22x	1534.09
6	1981.15	2.91x	1562.31
10	1573.26	3.67x	1792.22
14	1710.82	3.37x	1798.17
16	5874.92	0.98x	1691.49

TABLE 1 – Performances OpenMP pour le déplacement des fourmis.

3.2 Analyse OpenMP

L'accélération est significative et quasi-linéaire jusqu'à 10 threads, atteignant un pic de performance de **3.67x**. Cependant, à partir de 14 threads, les performances se dégradent, et avec 16 threads, le temps d'exécution devient équivalent à la version séquentielle. Cela s'explique par la contention matérielle (limite de cœurs physiques et hyperthreading) et la surcharge causée par les opérations `atomic` : trop de threads tentent de modifier la même zone mémoire simultanément.

Il est également intéressant de noter que le temps d'évaporation augmente légèrement avec le nombre de threads. L'évaporation est limitée par la bande passante mémoire (Memory Bound), l'ajout de threads crée donc un goulot d'étranglement sur le bus mémoire.

4 Parallélisation en Mémoire Distribuée (MPI)

Pour la parallélisation MPI, la première stratégie proposée a été implémentée : chaque processus conserve la carte entière mais ne gère qu'une fraction des fourmis. À la fin de chaque itération, une réduction globale (`MPI_Allreduce` avec l'opérateur `MPI_MAX`) est effectuée pour synchroniser la carte des phéromones entre tous les processus.

4.1 Résultats de Performance

Nombre de Processus	Temps Déplacement (ms)	Temps Évaporation / Sync (ms)
2	3287.85	3791.86
3	1963.10	3890.58
4	2701.99	5479.22
6	7219.13	8489.75
8	17792.30	14872.80

TABLE 2 – Performances MPI pour le déplacement et la synchronisation.

4.2 Analyse MPI

Le modèle MPI offre d'excellentes performances pour le calcul strict du déplacement (Compute Bound) jusqu'à 3 processus, abaissant le temps à 1963 ms.

Néanmoins, l'architecture révèle rapidement son point faible : **le coût des communications**. À partir de 4 processus, le temps d'exécution explose. Avec 8 processus, l'évaporation et la synchronisation prennent près de 15 secondes. Cela s'explique par la taille de la carte des phéromones. Lors de l'appel à `MPI_Allreduce`, une quantité massive de données (plusieurs mégaoctets par itération) doit transiter par le réseau ou la mémoire partagée du système, transformant l'application en un problème fortement limité par les communications (Network Bound).

5 Conclusion

Ce projet met en évidence les compromis classiques du calcul haute performance (HPC). OpenMP s'est révélé être l'approche la plus efficace pour ce problème précis grâce à un espace mémoire unifié évitant les lourdes copies de données. L'approche MPI pure, bien qu'efficace pour diviser la charge de calcul, est sévèrement pénalisée par le transfert de données globales. Une solution future pertinente serait une approche hybride (MPI + OpenMP), limitant le nombre de processus MPI pour réduire les communications, tout en maximisant les threads OpenMP par processus pour accélérer le déplacement.